AuthPilot: Autonomous Prior Authorization Agent

Oct 30, 2026 · @Asma

Executive summary

AuthPilot is an autonomous AI agent that owns the prior authorization process from the physician's order to the insurer's final decision, cutting a roughly 45-minute manual task to about 3 minutes of agent work plus one human approval click.

Prior authorization is the step where an insurer must approve a procedure or drug before the clinic can deliver it. Today, clinic staff gather chart notes, read payer policies, fill forms, chase responses and rewrite appeals by hand. AuthPilot reads the order and the patient chart, finds the matching payer policy, checks every criterion with a cited piece of evidence, requests whatever is missing, drafts the medical-necessity letter, submits it, and writes an appeal if the request is denied.

The hackathon MVP covers two clinical cases on fully synthetic data, a mock payer service, a live agent timeline dashboard, and an evaluation on a sealed test set with baselines, ablations and robustness tests. A human approves every submission, and every action is logged.

The problem

Prior authorization is a slow, document-heavy loop that pulls clinical staff away from patients and delays care.

A typical request goes through the same manual steps every time:

- Find out whether the payer requires authorization for this procedure or drug
- Locate the current payer policy and read its criteria
- Dig through the chart for notes, imaging, labs and therapy history
- Fill the payer form and write a medical-necessity letter
- Submit, wait, call for status, and rewrite the request after a denial

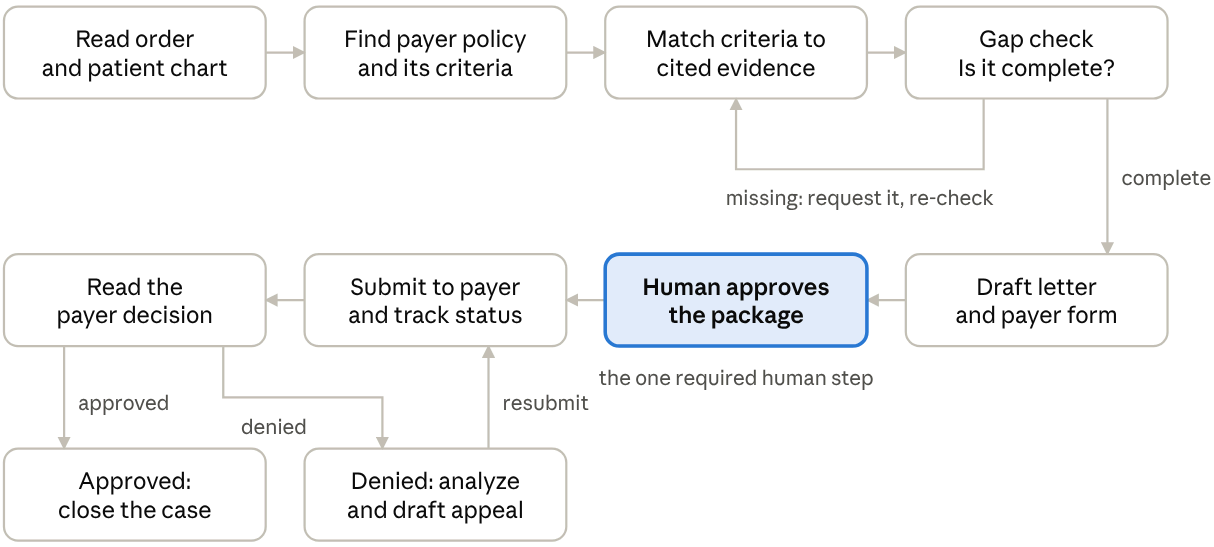
Three things make this costly. Policies are long, differ by payer and change often. Evidence is scattered across the chart, so staff miss items and get denials for incomplete files. And every denial restarts the work. The result is days of delay for the patient and hours of administrative time for the clinic.

Assumption to validate with a mentor: the 45-minute figure used in this proposal is an estimate for a single request, not a measured value. Replace it with a number from a clinic or hospital admin during the hackathon.

The solution

AuthPilot is one agent that owns each request from the physician's order until the payer decides. It chooses its next step from the state of the case, and it loops by itself when evidence is missing or a request is denied.

The agent loops on gaps and denials; a person approves once



agent workflow · 2 loops, 1 human gate

Missing evidence sends the case back for another check, and a denial triggers an appeal that is resubmitted. The approval step is the only point where a person must act.

Fit with the challenge

AuthPilot takes ownership of a full business process rather than answering questions about it, and it combines the three example use cases from the brief.

Challenge theme	How AuthPilot uses it
Contract review	Compares the patient chart against payer policy clauses, flags unmet criteria and suggests fixes
Employee onboarding	Creates tasks and requests for missing documents, then tracks them to completion
Sales development	Drafts personalized, evidence-based letters and follows up on submissions until a decision

The agent shows autonomy in four ways:

- It decides what to do next from the state of the case, not from a fixed script
- It loops when evidence is missing and when a request is denied
- It uses tools: chart lookup, policy search, form filling, submission and status check
- It stops only at one deliberate human gate, the final approval before submission

MVP scope

The MVP proves one narrow workflow end to end on two clinical cases, using synthetic data only.

Clinical cases

Case	Example criteria the agent checks	Demo outcome
Lumbar spine MRI	Failed conservative therapy for at least 6 weeks, no red-flag symptoms, documented exam	Approved after the agent fills a missing-evidence gap
High-cost drug (for example a biologic or GLP-1)	Qualifying diagnosis or BMI, prior therapy tried and failed	Denied first, then approved after an agent-written appeal

The criteria above are illustrative. The policies are written by the team as realistic 3-4 page PDFs, so no real payer documents are copied. A third procedure with its own policy is built only as an unseen test of adaptation, not as a finished feature.

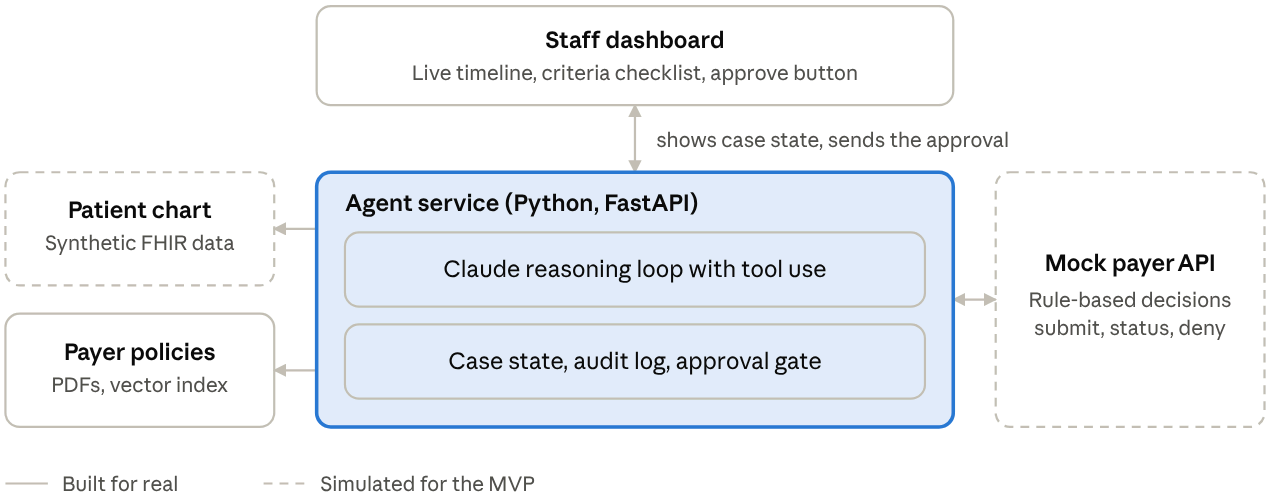
Built for real vs simulated

Built for real	Simulated
Agent reasoning loop with tool use	Payer portal (a small mock API with rule-based decisions)
Policy ingestion and criteria extraction	EHR (synthetic FHIR patient records)
Evidence matching with chart citations	Fax and email delivery (written to a log)
Gap detection and missing-information requests	
Letter and appeal drafting	
Approval dashboard and audit trail	

Technical architecture

A single agent service coordinates four things: the patient chart, the payer policies, the payer, and the staff dashboard.

One agent service ties the chart, policies, payer and dashboard together



architecture · agent service with 4 connected parts

The agent calls the chart, policy index and payer through tools, and the dashboard reads the same case state and audit log.

Layer	Choice	Role
Agent model	Claude via the API with tool use	Plans each step and decides when to loop
Orchestration	Python, FastAPI, optionally LangGraph	Case state, retries and the approval gate
Patient data	Synthea-generated FHIR records, SQLite or Postgres	Synthetic charts with known answers
Policy retrieval	Chroma or pgvector	Finds the right policy section for each criterion
Mock payer	Small FastAPI service	Receives submissions, returns approve or deny with reasons
Frontend	React or Next.js (Streamlit if time runs short)	Live agent timeline, checklist, approval click

Agent tools: `get_patient_record`, `search_policy`, `check_criteria`, `request_missing_info`, `draft_letter`, `submit_request`, `check_status`, `draft_appeal`.

Safety, privacy and trust

AuthPilot is designed so staff can verify every claim the agent makes and stop it at any point.

- **Human approval gate.** Nothing is sent to the payer until a staff member approves the final package.
- **No invented facts.** Every criterion result must cite a span of the chart or policy. If no evidence is found, the agent says "evidence not found" and never writes the claim into a letter.
- **Structured outputs.** Criteria matching returns a fixed schema, and any result without a source span is rejected before it reaches the user.
- **Audit trail.** Every tool call, decision and reason is logged with a timestamp and can be replayed in the dashboard.
- **Synthetic data only.** All patients are generated, so the hackathon build holds no real health information.

- **Privacy by design.** Data minimization, role-based access and encryption are part of the design. Saudi PDPL requirements and HIPAA-style principles will be reviewed with a compliance mentor before any pilot.
- **Clinician stays accountable.** The agent prepares and recommends. The clinical and billing decision remains with a person.

Success metrics and evaluation

Results are reported on a sealed test set that is never used for tuning, with a baseline, ablations and robustness tests alongside.

A seeded script generates 60 synthetic cases: 40 for development and 20 for a sealed test set made with a different seed. Each case has a known correct answer: which criteria are met, which evidence is missing, and whether the payer should approve. The targets below are goals, not results already achieved.

Metric	How it is measured	Target
Criteria-match accuracy	Agent verdict per criterion vs the labeled answer, on the sealed test set	90% or higher
Final decision accuracy	Agent's approve or needs-evidence call vs the labeled outcome	90% or higher
Citation validity	Share of cited evidence spans that exist in the chart and support the claim	100%
Missing-evidence detection	Gaps found by the agent vs gaps planted in the case	85% or higher
Robustness drop	Accuracy on perturbed cases vs the same cases unperturbed	5 points or less
Denial recovery	Denied cases that the agent appeals correctly	70% or higher
Time per request	Agent run time vs the estimated manual baseline	Under 5 minutes

Every run is saved with its seed, model version and prompt version so any number in the slides can be reproduced. Failures are shown and analyzed, not hidden.

Robustness and adaptation

The agent reads the policy instead of hard-coding rules, so changing the inputs should change its answers correctly with no code changes. Each test below is run on the same cases before and after the change, and the accuracy drop is reported.

Change to the inputs	What changes	Expected behavior
Policy threshold	For example 6 weeks of therapy becomes 8 weeks	Verdicts flip only for the affected cases
Reworded policy	Same criteria, different wording and layout	Same verdicts
Noisy chart	Notes reordered, abbreviated or padded with irrelevant text	Same verdicts, citations still valid
Missing evidence	A note carrying key evidence is removed	The agent flags a gap and does not guess
Unseen procedure	A third procedure and policy never used in development	The agent handles it from the new policy PDF alone

Baselines, ablations and iteration

The agent is compared with simpler alternatives to show that each part earns its place.

Variant	What is removed or changed	Question it answers
Single-prompt baseline	One prompt with chart and policy together, no tools or loops	What does the agent add over the simple approach?
No retrieval	Full policy placed in the prompt instead of retrieved sections	Does retrieval help accuracy and cost?
No citation check	Evidence spans are not validated	How much does validation reduce invented evidence?
No gap loop	The agent never asks for missing evidence	Does the loop improve final decisions?

Variant	What is removed or changed	Question it answers
No appeal loop	Denials end the run	How many denials does the appeal recover?

Iteration follows a fixed routine: change one thing, run the development set, then the sealed set, and record both. A regression check fails the run if any score drops against the last saved result. Time and token cost per request are reported next to accuracy, so trade-offs are visible.

Engineering quality

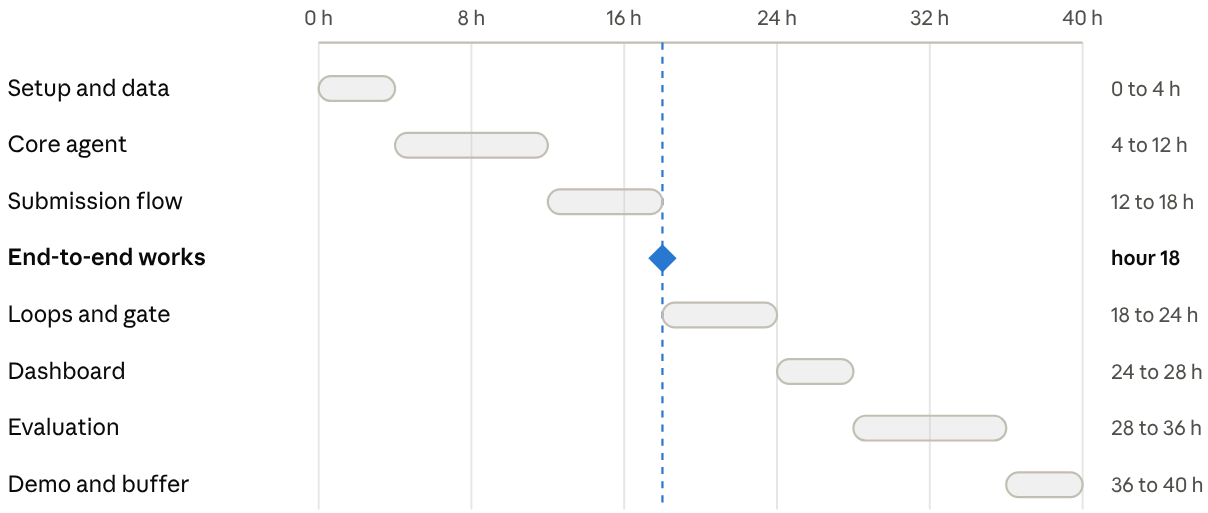
- One command to start the demo and one to run the full evaluation
- Pinned dependencies, fixed seeds and a Docker setup so another person can reproduce the results
- Typed schemas for every tool input and output, and unit tests for each tool
- A README covering setup, architecture, results and known limitations

40-hour delivery plan

The plan puts a plain but working end-to-end flow in place by hour 18, so the remaining time goes to evaluation, a plain dashboard and the demo. Evaluation gets 8 hours, taken from dashboard polish, because judges score measured results on unseen cases.

A working end-to-end flow by hour 18 leaves the second half for polish

40-hour solo build, hours from the start



delivery plan · 7 phases over 40 hours, 1 milestone

The dashed line marks the milestone: if the plain flow is not working by hour 18, scope is cut before anything new is added.

Phase	What gets built
Setup and data	Scope check with a mentor, repository, synthetic patients, two policy PDFs
Core agent	Order intake, policy retrieval, criteria extraction, evidence matching with citations
Submission flow	Gap detection, letter drafting, mock payer API, submission
Loops and gate	Denial and appeal loop, human approval gate, audit log
Dashboard	Plain live agent timeline, criteria checklist, approval view
Evaluation	Seeded development and sealed test sets, baseline and ablation runs, robustness tests, regression check, Arabic as a stretch goal
Demo and buffer	Demo script, slides, rehearsal, bug fixes, backup recording

Risks and fallbacks

- **Scope creep:** cut Arabic first, then extra dashboard views, and keep the two clinical cases.
- **Invented evidence:** structured outputs, and any result with no source span is rejected.
- **Demo failure:** a pre-recorded backup video and a cached run that can be replayed.
- **Domain gaps:** use mentors early for the real clinic workflow, NPHIES alignment and the privacy framing.

Impact and roadmap

Faster authorization means patients start treatment sooner and clinic staff spend their time on care instead of paperwork.

Impact

- Patients: fewer days waiting for approval of scans, drugs and procedures
- Clinics: less staff time per request and fewer denials caused by incomplete files

- Payers: cleaner, better-documented requests that are easier to review

Saudi context. Saudi Arabia runs a national health information exchange, NPHIES, built on FHIR. The submission format in the MVP will be modeled to align with it, and the exact message structure will be confirmed with a health-IT mentor before it is claimed in the demo. Arabic and English support for letters and the interface is a stretch goal.

After the hackathon

1. Pilot with one clinic using de-identified data and a compliance review
2. Add more procedures and payer policies, with automatic policy-change monitoring
3. Connect to real EHR and payer interfaces behind the human approval gate
4. Learn from outcomes to predict approval likelihood and improve letters